

# SOLARI

## System Architecture Document

**SolarWise AI: Intelligent Solar Engineering Assistant**

Compliant with 2026 NEPRA Net-Billing Regulatory Framework

Generated: August 2026

# 1. Project Overview

---

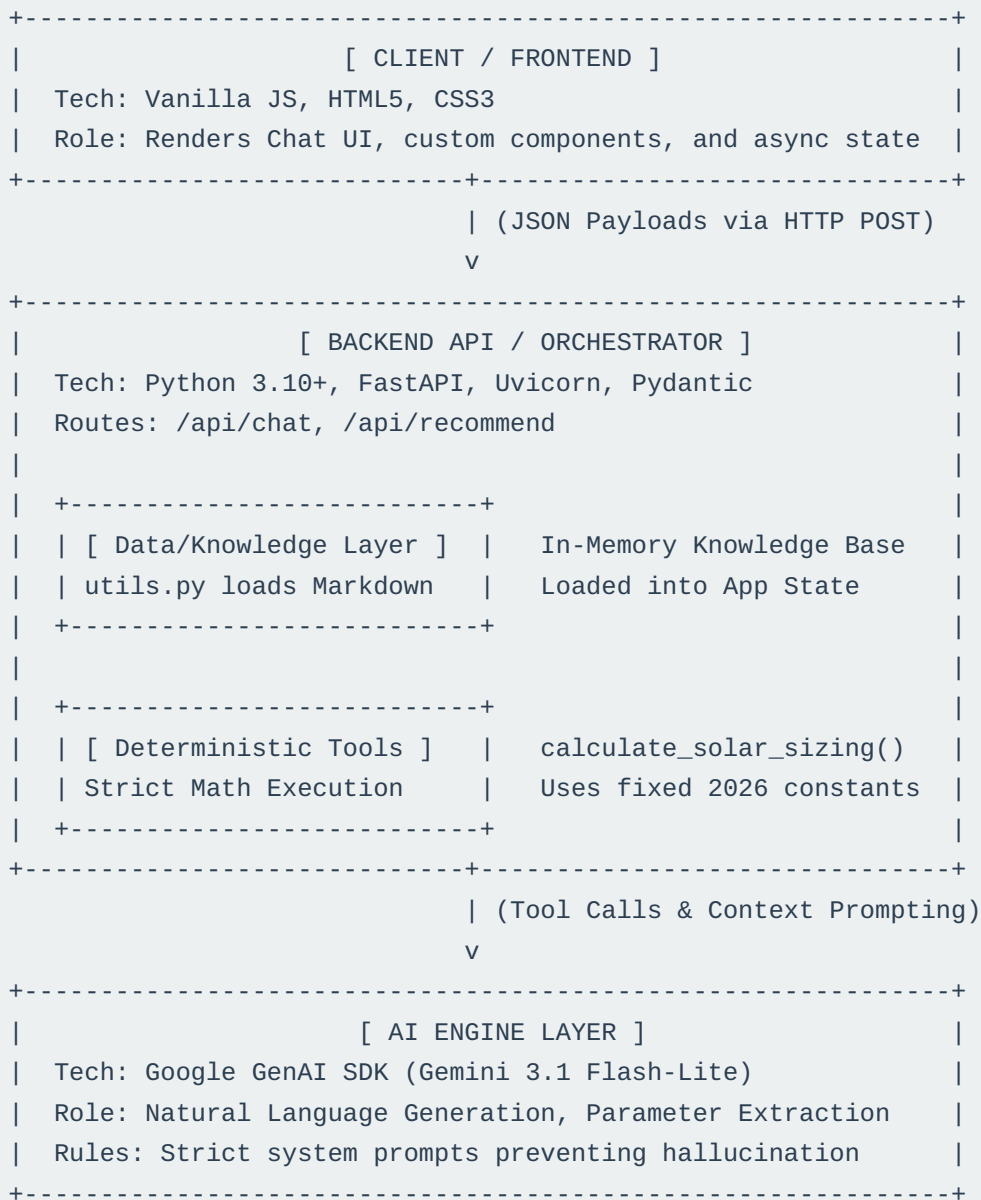
Solari (SolarWise AI) is an advanced, expert-level Solar Industry Engineering Assistant chatbot. It is explicitly designed to cater to residential, commercial, and industrial solar photovoltaic (PV) solutions in Pakistan, operating strictly under the updated 2026 NEPRA Net-Billing Regulatory Framework.

The core architectural philosophy of the Solari system is the seamless convergence of natural language AI consultation with deterministic, rigid mathematical sizing. By ensuring that mathematical operations are handled via programmatic backend functions rather than probabilistic Large Language Model (LLM) generations, Solari eliminates the risk of AI hallucinations regarding pricing, electrical specifications, and solar yield projections.

## 2. System Architecture Diagram

---

The following text-based representation outlines the data flow, tier separation, and architectural topology of the Solari platform:



### 3. The Frontend Layer

The frontend is designed to be lightweight, incredibly fast, and entirely devoid of complex framework dependencies. This allows for seamless, frictionless integration into existing corporate platforms or standalone web deployments.

## 3.1 Core Technologies

- **Tech Stack:** Vanilla JavaScript, HTML5, and CSS3.
- **Communication:** Standard Fetch API managing asynchronous requests and state updates.

## 3.2 Functionality & Rendering Workflow

The user interface provides a sleek, responsive chat experience. When a user submits a query, the frontend captures the input and constructs a standardized JSON payload, securely passing it to the server:

```
Example Payload: {"message": "I need a system for a 50,000 PKR bill",  
"system_type": "on-grid"}
```

The UI maintains visual feedback by rendering typing indicators while awaiting the backend's response. Once the server responds, the frontend asynchronously maps the returned data .message into the conversation thread.

Crucially, if the backend payload includes a data .tool\_result object (indicating that a deterministic mathematical calculation was performed), the frontend dynamically builds rich custom UI cards. This transforms complex calculations into clean, visual representations—displaying recommended system size, panel count, and inverter specifications cleanly, rather than relying on dense text blocks.

# 4. The Backend API Layer

---

The backend serves as the central brain and orchestrator of the Solari application, managing system state, data flow, and cross-layer integration between the UI and the AI.

## 4.1 Core Technologies

- **Framework:** Python 3.10+ utilizing the FastAPI framework for asynchronous, high-performance routing.
- **Server:** Uvicorn ASGI server to handle high concurrency and web traffic efficiently.
- **Validation:** Pydantic for rigid schema enforcement and payload validation.

## 4.2 Routing & Endpoints

| Endpoint       | Method | Primary Function                                                                                                                                                                            |
|----------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/chat      | POST   | The primary conversational endpoint. Accepts user text, maintains conversational state, triggers the AI engine, processes tool callbacks, and returns the final response.                   |
| /api/recommend | POST   | A direct, deterministic endpoint for calculating solar sizing. Bypasses the AI engine entirely, making it ideal for standard UI forms or third-party integrations requiring immediate math. |

The backend also robustly handles Cross-Origin Resource Sharing (CORS) configurations, ensuring seamless, unblocked, and secure communication with the frontend client.

## 5. The Data / Knowledge Layer

To maximize speed, minimize infrastructural overhead, and reduce operational costs, Solari explicitly avoids complex external Vector Databases in favor of a highly optimized In-Memory Knowledge Base architectural pattern.

### 5.1 Architecture & Data Sources

During the FastAPI server boot sequence (leveraging the asynchronous `lifespan` event hook in `main.py`), a dedicated utility module (`utils.py`) dynamically reads the latest local research files from the filesystem. These comprehensive Markdown files include:

- `hardware_research.md`: Detailed specifications on Tier-1 panels, inverters, and structural mounting hardware.
- `operations_support.md`: Strict guidelines on O&M (Operations & Maintenance), warranties, and diagnostic troubleshooting.
- `systems_research.md`: Exhaustive net-billing rules, financial mechanics, and system sizing methodologies under NEPRA regulations.

### 5.2 Performance & Context Injection

Once read into memory, these documents are concatenated into a unified string and securely injected into the FastAPI application state. When a user query is received, this comprehensive knowledge context

is instantaneously available to build the AI's prompt window. This approach achieves true zero-latency data retrieval.

## 6. The AI Engine Layer

---

The AI Engine layer is powered by the Google GenAI SDK. It provides the natural language reasoning required to consult users while being strictly bound by predefined system guardrails.

### 6.1 Core Technologies & System Prompting

- **Model:** Gemini 3.1 Flash-Lite, chosen for its fast inference speeds and low latency in chat applications.
- **System Guardrails:** A highly restrictive system prompt (`system_prompt.txt`) is enforced globally via the `GenerateContentConfig` configuration.

The system prompt mandates strict adherence to the **2026 NEPRA Net-Billing Regulatory Framework**, enforces rigid safety guidelines (explicitly forbidding the AI from providing instructions for DIY high-voltage wiring), and instates a zero-tolerance policy against hallucinating prices, specs, or yield metrics.

### 6.2 Native Tool Calling (Function Calling)

To fundamentally prevent the AI from attempting to do complex mathematics—a common source of LLM hallucination—the backend defines a strict `TOOL_SCHEMA` dictionary outlining a function named `calculate_solar_recommendation`.

The execution workflow is as follows:

1. When the Gemini AI detects that a user is asking for a sizing estimate or mathematical advice, it immediately pauses generation and yields the extracted parameters (function call arguments) back to the FastAPI server.
2. The server natively executes the deterministic Python function using strict, hardcoded 2026 industry constants (e.g., **550W panels, an 80% Performance Ratio, and 4.5 Peak Sun Hours**).
3. The precise mathematical result is returned to the AI engine, empowering it to formulate a conversational response completely grounded in hard facts.
4. The raw JSON output from the calculation is appended to the main API response, allowing the frontend to simultaneously render a visual data card alongside the AI's text.